

Endpoint Investigator — Documento Técnico

Disciplina: Tecnologias Hackers · Prof. Rodolfo Avelino · Avaliação Intermediária **Grupo:** Enzo Christofolletti, Thomas Rudge e Thomas Kassabian · **Apresentação:** 6 de outubro

1. Problema

Um analista que recebe um endpoint Linux precisa decidir **o que merece investigação**. As ferramentas usuais (ps, systemctl, ls -l) entregam listas independentes; o risco, porém, está nas **relações** entre elas. Um serviço rodando como root não é vulnerabilidade. Um script com permissão 0777 também não é, por si só. A combinação "serviço root + executa o script + qualquer usuário pode alterá-lo" é uma **relação de privilégio insegura**. Mesmo ela **não prova exploração**.

O Endpoint Investigator coleta um snapshot sob demanda (processos, permissões, serviços e journal), correlaciona as fontes e entrega, para cada caso, quatro coisas distintas:

Elemento	Significado
Evidência	o que foi observado, com a origem exata
Interpretação	o significado técnico dessa evidência
Hipótese	explicações concorrentes, comparadas formalmente
Evidência ausente	o que falta para confirmar ou rejeitar a hipótese

Quando as evidências não bastam, a ferramenta responde **INCONCLUSIVO** em vez de adivinhar.

2. Estratégia de investigação

A estratégia combina três ideias, cada uma respondendo a uma exigência do enunciado:

- Grafo de proveniência.** Processos, serviços, arquivos, diretórios, usuários, eventos de log e destinos remotos viram nós; PPID, "roda como", "executa", "usa script", "dentro do diretório", "registrado por" e "conecta a" viram arestas. Isso impede tratar as dimensões como listas independentes.
- Regras que cruzam fontes, com pré-requisitos.** Sete regras (R1 a R7) consultam o grafo. Algumas só existem se outra disparou, o que reduz falsos positivos.
- Matriz ACH (Analysis of Competing Hypotheses, Heuer).** Compara hipóteses por refutação e define, de forma objetiva, quando o resultado é inconclusivo, qual a confiança e qual a evidência ausente.

A análise é **orientada por contexto**: só examinamos arquivos usados por processos e serviços e seus diretórios pai, além de uma busca restrita (setuid em /usr/local, /opt, /home, /tmp; arquivos graváveis por todos em /opt e /usr/local). Não há varredura do filesystem inteiro.

3. Arquitetura

Segue o fluxo de referência do enunciado (coleta → normalização → correlação → evidências → hipóteses → resultado):

COLETA → NORMALIZAÇÃO → GRAFO DE PROVENIÊNCIA → REGRAS R1-R7 → GRAFO DE CENÁRIO → ACH → OCSF/terminal

dataset|live campos ECS entidades e relações TOML + Python pré-requisitos C/I/N

- Coleta (dois coletores, mesma saída).** dataset lê os quatro artefatos do gerador do professor; live lê /proc, systemctl show, os.stat e journalctl -o json. Daí em diante o motor é um só: o que se testa nos datasets é o que roda ao vivo. O SHA-256 dos artefatos entra no relatório (cadeia de custódia).
- Normalização.** Transforma texto em fatos: /bin/bash /opt/backup/backup.sh vira interpretador mais script (é isso que liga serviço a arquivo); o modo 0777 vira flags (world_writable, setuid, sticky...); backup-agent[2417]: ... liga a linha de log ao PID; curl https://host/... vira destino remoto. Os campos seguem o ECS e cada fato guarda sua origem (permissions.csv:3).
- Regras.** Cada regra é um arquivo TOML (id, severidade, interpretação, o que não foi provado, evidências

ausentes e **falsos positivos conhecidos**, no estilo Sigma) mais um matcher Python. Os falsos positivos declarados alimentam a seção de limitações.

- **Saída.** Terminal legível e JSON em **OCSF Detection Finding (classe 2004)**; interpretação, hipóteses e matriz ACH vão num bloco de extensão investigation. Severidade (impacto se verdadeira) e confiança (força da evidência) são eixos separados.

Stack: Python 3.10+, **somente biblioteca padrão**, cerca de 1.900 linhas (1.500 sem a camada de LLM).

4. Principais correlações

Regra	Fontes cruzadas	Resultado
R1 Relação de privilégio insegura	serviço + processo + permissão	serviço root usa recurso que não-root altera ou substitui. Verifica o arquivo e todos os diretórios até a raiz : script 0700 em diretório 0777 pode ser apagado e recriado. Descarta diretório com sticky bit
R2 Contexto de execução	processo + cadeia de PPID + usuário + journal	processo root filho de processo de usuário comum, sem sudo/su registrado
R3 Linha do tempo	serviço + log + sessão + mtime	usuário comum presente enquanto existe recurso alterável usado por root. Pré-requisito: R1
R4 SUID fora do padrão	permissão + caminho	setuid de root em diretório não padrão; severidade maior se o nome consta no GTFOBins
R5 Saída de rede privilegiada	processo filho + serviço root + URL	curl/wget de serviço root: inconclusivo , não é C2 por si só; sobe de severidade se R1 marcou o script
R6 Configuração inadequada	permissão sem consumidor	arquivo gravável por todos que ninguém usa
R7 Uso legítimo de privilégio	serviço root + script protegido	registra "verificado e correto", prova que não aplicamos "root = vulnerável"

Correlações pedidas no enunciado: processo + serviço + permissão (R1); processo + PPID + usuário (R2); serviço + arquivo + usuário (R1); processo + serviço + log (R3, mais linhas de log em R1 e R5).

Matriz de correlação entre fontes. Cada regra declara em seu .toml quais fontes cruza, e a ferramenta deriva daí a matriz (--matrix): capacidade (o que pode ser correlacionado) e o que foi de fato correlacionado no snapshot. As chaves de ligação são: processo × serviço (comando = ExecStart, MainPID, PID no journal); processo/serviço × permissão (script ou binário = caminho do arquivo); processo × log (PID); serviço × log (nome do serviço); permissão × log (mtime × horário do evento). --graph e --graph-dot mostram o grafo de proveniência.

Matriz ACH, exemplo (cenário correlation, R1). Hipóteses: H1 legítimo, H2 má configuração explorável sem exploração observada, H3 exploração já ocorreu.

Evidência	H1	H2	H3
serviço executa como root	C	C	C (não diagnóstica)
backup.sh com modo 0777	I	C	C
backup terminou com status=0K	C	C	N
sessão SSH do aluno ativa	N	C	C
mtime do script anterior ao login	N	C	I (frágil)
Inconsistências	1	0	1

Veredito **H2**, confiança Medium (margem 1). A sensibilidade informa que a conclusão **depende do mtime**: sem ele, H2 e H3 empatam. Faltam hash contra baseline e auditd. Sem sessão de usuário comum (cenário writable_parent), H2 e H3 empatam em zero inconsistências e o veredito é **INCONCLUSIVO**.

Confiança: inconclusivo = Low; margem 1 = Medium; margem ≥ 2 = High. **Sensibilidade:** cada evidência não essencial é retirada e o veredito recalculado; se muda, o relatório diz de qual evidência a conclusão depende.

5. Decisões técnicas

Decisão	Motivo
Mesmo motor para dataset e ao vivo	O que se testa é o que se demonstra
ACH em vez de pontuação aditiva	Define objetivamente "inconclusivo", confiança e evidência ausente
Regras como dados (TOML)	Cada regra declara os próprios falsos positivos
Verificar todos os diretórios até a raiz	A checagem ingênua (só o modo do script) perde a substituição via diretório gravável
timestamp de processes.csv tratado como hora da coleta	No dataset ele marca a ordem da linha, não o início do processo; início de sessão vem do journal
Log ligado ao processo pelo PID	O identificador do log nem sempre coincide com o executável (ex.: log com o PID do curl)
Symlinks seguidos com os.stat	O modo 0777 do próprio link (/bin, /lib) gerava falsos positivos ao vivo
LLM opcional e verificada	A análise é determinística e reproduzível; a LLM só explica findings prontos, e o que não tem evidência é descartado
Python 3.10+ com fallback de TOML	Compatibilidade com o ambiente de desenvolvimento e com a Kali

Gerador de datasets. Adaptamos o gerador do professor: random_noise funciona; o cenário de permissão grava o nome correto no metadata; horários com fuso real; cenários novos (writable_parent, user_root_process); metadata.json com expected_findings; opções --scenario, --batch e --seed.

6. Resultados e avaliação

tools/evaluate.py --seeds 12 gera lotes (8 cenários × 12 seeds = 96 datasets), roda a ferramenta e compara com o gabarito. Resultado: **precisão e recall 1,00 para R1 a R7** e todos os vereditos ACH esperados atingidos. A seed só varia o cenário random (nome do serviço, PIDs, modo do script); os outros sete são fixos. Os 96 datasets correspondem, portanto, a **19 conjuntos distintos** (7 fixos + 12 variações de random).

Como interpretar: os findings esperados vêm do gerador; o gabarito de **vereditos** foi derivado por nós do raciocínio de cada cenário. O número mede coerência do motor com esse raciocínio, não verdade absoluta, e **não prova generalização**: com poucos conjuntos distintos, o teste confirma a lógica, não a robustez estatística.

Modo ao vivo na Kali. Em laboratório controlado (/opt/lab, com script de serviço, setuid inofensivo, arquivo 0777 órfão e curl para um IP que não responde), verificamos:

Situação	Resultado
Linha de base	só R4 em chrome-sandbox (falso positivo conhecido)
Script 0777	R1, R4, R5 (Medium), R6
Script 0700	R1 some; R7 (legítimo); R5 cai para Low
Script 0700 em diretório 0777	R1 volta, via diretório pai
Diretório com sticky bit	R1 some (falso positivo descartado)
Sem o bit setuid	R4 do laboratório some

Os nomes do laboratório não existem no código: os resultados mudam só com as permissões, o que mostra que as regras operam sobre fatos do sistema e não sobre valores fixos.

7. Uso de IA

Ferramentas de IA (Claude) foram usadas **no desenvolvimento**: apoio à programação, à documentação, à revisão do desenho e à explicação dos resultados, como o enunciado permite. Em **tempo de execução**, coleta, normalização, correlação, hipóteses e veredito são produzidos por código determinístico. Há uma camada de LLM **opcional** (--llm ou ENDPOINT_LLM=on no .env; API da OpenAI) que apenas redige a explicação de findings já prontos. Ela recebe só os findings (nunca os dados crus), devolve JSON com schema fixo em que toda afirmação cita um id de evidência, e um **verificador** descarta o que cita evidência inexistente, menciona caminho/IP/PID ausente das evidências ou afirma exploração sem base. A LLM não altera classificação nem veredito; a discordância aparece como segunda opinião e o relatório informa a taxa de afirmações sem suporte. Sem chave ou sem rede, a ferramenta segue só com a análise determinística. A IA não é a camada de análise.

8. Limitações

Escopo. Snapshot, não monitoramento. Não analisamos o conteúdo de scripts e binários nem temos baseline de hash (aparece como evidência ausente). O mtime pode ser forjado e os horários do dataset não têm fuso confiável, então a linha do tempo é aproximada.

Método. As marcações C/I/N do ACH são **codificadas por nós, por regra**; no método original quem marca é um analista. O HOLMES foi adaptado: usamos só as ideias estruturais (duas camadas, pré-requisitos, produto ponderado), não o processamento de logs de auditoria em tempo real. R3 e R4 não têm matriz ACH.

Fontes não cobertas. Conexões de rede e portas em escuta (o exemplo "processo gerenciando uma porta" do enunciado), arquivos abertos, usuários e grupos, persistência (cron, timers) e hash dos arquivos analisados. Como o snapshot não vê cron nem timers, "arquivo sem consumidor" (R6) é evidência frágil.

Tratamentos ainda ausentes. Não detectamos evidência conflitante entre fontes (no cenário `privileged_service` o journal diz "Deactivated" e o serviço aparece como `running`). `DynamicUser=` e `RootImage=` não são coletados. Serviço sem processo associado só aparece como evidência ausente em R1.

Modo ao vivo. Apenas serviços em execução; journal limitado às últimas 1000 linhas; sem root a visão é parcial; a busca de `setuid` pode levar de 30 s a alguns minutos.

Camada LLM. O verificador descarta invenções (evidência inexistente, caminho, IP ou PID ausentes), mas **não detecta erros de raciocínio**: em teste, um modelo pequeno discordou do veredito com um argumento incorreto. Por isso a LLM nunca altera o veredito e sua saída é tratada como texto auxiliar, enviado a um serviço externo (os findings, não os dados crus).

Falsos positivos conhecidos. R1: sticky bit e permissões acima do coletado. R2: `sudo/su` com log não coletado. R4: `setuid` legítimo de terceiros (ex.: `chrome-sandbox`). R5: checagem de status ou métricas legítimas. R6: consumidor fora do snapshot. R7: conteúdo malicioso apesar de permissões corretas. A lista completa está nos arquivos TOML de cada regra.